

## Final Project Report

student's name: Alaa Kalla,

student number: 102633113

degree program: computer science

year of studies 2

date 28.04.2025

### **General Description**

The project is a calendar application developed in ScalaFX.

It allows users to create, view, and manage events on a daily, weekly, monthly, and yearly basis.

The application also supports smart event searching based on name, category, or description, reminders through local audio and email notifications, and exporting events to a standard ICS (iCalendar) format.

Compared to the initial project plan, the overall direction remained the same. However, some advanced features like editing existing events and detailed listing of events under each day in the weekly view were not fully completed due to time limitations.

The project was implemented at a moderate difficulty level because of the wide range of features and the integration of external libraries (ScalaFX, JavaMail).

### **User Interface**

the user can

View a calendar in Year, Month, Week, or Day modes.

Add new events by clicking a day or dragging across hours in Day view.

List upcoming events automatically.

Search events by entering keywords from event name, category, or description.

Toggle dark mode for a different visual appearance.

Export events into a .ics file using the Export button.

See Finnish public holidays automatically highlighted.

User commands are executed by clicking buttons (e.g., Add Event, Export, etc.) or interacting directly with the calendar.

## **Program Structure**

The program is organized into several classes and files:

Main (CalendarApp.scala): Manages UI layout and user interactions.

Event.scala: Defines the Event case class that models a calendar event.

Calendar.scala: Manages adding, removing, listing, and saving events.

ReminderUtils.scala: Handles playing audio and sending email reminders.

UIUtils.scala: Provides helper methods for UI coloring and highlighting.

HolidayUtils.scala: Provides logic for retrieving Finnish holidays.

### **Class relationships**

CalendarApp interacts with Calendar to manage event data.

Calendar stores a list of Event objects.

ReminderUtils is called when scheduling or sending reminders.

UIUtils helps with button styles and coloring based on event categories.

HolidayUtils provides holiday dates and descriptions.

## **Algorithms**

The following algorithms were used:

Sorting events: Events are sorted by their date and time for easy listing.

Reminder scheduling: Calculates the delay between the current time and the event's start time and sets a Java TimerTask.

Event search: Simple string matching is performed across event names, descriptions, and categories.

Holiday highlighting: When building the calendar, dates are checked against a list of holidays and colored differently if a match is found.

The algorithms are simple but efficient for a desktop application.

Alternative solutions could include using more complex scheduling libraries for reminders, but current methods were sufficient for the project scope.

## **Data Structures**

Scala Collections (`mutable.Buffer`): Used to store the list of Event objects.

`Map[LocalDate, String]`: Used to store holiday dates and their descriptions.

Data structures were chosen to keep operations simple, quick, and mutable because events can be added and removed frequently.

### **Files and Internet Access**

Saving Events: Events are stored in a text file locally for persistence.

Exporting Events: Events can be exported into .ics files following the iCalendar standard.

Internet Access: SMTP via JavaMail was used to send reminder emails.

Connection uses standard secure SMTP over TLS.

## **Testing**

Testing was done through:

Manual GUI testing during development.

Separate unit tests:

`EventTest.scala` tested duration calculation.

`CalendarTest.scala` tested adding and removing events.

`ReminderUtilsTest.scala` tested reminder scheduling logic.

Testing was carried out regularly after each implemented feature.

The test coverage is good for the critical parts, but some UI-specific parts (e.g., mouse interactions) were tested manually only.

#### Known Bugs and Missing Features

Editing existing events was not implemented.

Weekly view displays days and events separately but is not fully optimized (it shows only basic event listing for each day).

No database support; data is only saved in text format.

Planned improvements if time allowed:

Allow users to modify event details.

Improve week view to group events more visually under each day.

Auto-backup system for event files.

### **3 Best Sides and 3 Weaknesses**

#### Best Sides

1. Rich Features: Many useful features were implemented including event creation, smart search, reminders, and export.
2. Good Program Structure: Classes are separated logically, making the code relatively modular and easy to extend.
3. User-Friendly Interface: UI is simple, clean, and functional with minimal training required.

#### Weaknesses

1. Incomplete editing feature: Event editing functionality is missing.
2. Basic Weekly View: Could be visually improved to better organize multiple events under each day.
3. Manual Reminder Testing: Some parts like audio reminders needed manual confirmation rather than full automation.

#### Deviations from the Plan

Initially, the plan was to complete all features including full event editing and advanced visual weekly view.

However, due to the time limitations, priorities were shifted towards delivering a stable base application with core features working well.

The timeline needed some extensions around the mid-project phase to ensure event search and export features were finished properly.

## **Final Evaluation**

Overall, the project was successful.

It provides a useful calendar application that meets the original goals while still leaving room for further improvements.

The structure allows adding more features later easily (notifications, database storage).

If starting again, I would focus earlier on event editing features and finalize the week/day views earlier before adding extras.

### **References and External Code**

JavaMail documentation was used to understand sending emails.

ScalaFX documentation and small examples were referred to for UI design.

AI tools (ChatGPT,Grok) were used during early planning and occasionally to help debug unclear errors, but all final code was manually written by the student.

I have seen websites about scala project in my own language so It helped me with build the codes and having information about doing calendar project in scala

No code was copied from external projects.

Any AI or external help was used only for explanation or debugging assistance, not code writing